

MVP BUILD BLUEPRINT

ITC-Rec Engine

System architecture, reconciliation workflow, and ER / data model for the first testable build

This pack is the build map for a thin, end-to-end MVP. The goal is one working slice you can test with real data: upload a Purchase Register and a GSTR-2B file, run the multi-pass match, see the buckets, review the edge cases, and export a reconciliation report. Heavy enterprise integrations (live ERP and GSTN pulls, multi-channel vendor outreach, a vendor portal) are deliberately deferred so you can prove the core matching value first.

MVP scope

In scope — build now	Deferred — post-MVP
Email/password auth · single organization	Live ERP / Tally / Zoho pull (use file upload for now)
Manual upload: Purchase Register (CSV/XLSX) + GSTR-2B (JSON/XLSX)	Live GSP / GSTN 2B API + OAuth (download from portal)
Parse + normalize (GSTIN, invoice-no, dates, tax heads)	WhatsApp / SMS notice channels (email/PDF only)
Multi-pass matching engine (exact → normalized → tolerance → fuzzy)	Vendor self-service correction portal
Result buckets + analyst review / force-match	Multi-tenant / multi-GSTIN at scale
Discrepancy notice (PDF) + reconciliation export (XLSX/PDF)	Automated Sec 16(4) / Rule 37 timers (manual flag for now)
Basic tamper-evident audit log	SSO, role hierarchies, billing

How to read the diagrams

- **Rounded blue box** = a screen, service, or process step you build.
- **Cylinder** = database (PostgreSQL); **folder** = file storage.
- **Dashed amber / grey** = optional in MVP or a post-MVP integration.
- **Diamond** = a decision; **ellipse** = start / end of the flow.
- **ER tables**: navy header = entity, **PK** = primary key, **FK** = foreign key, and **1 · N · 0..1** mark cardinality.

1. System Architecture — the connections

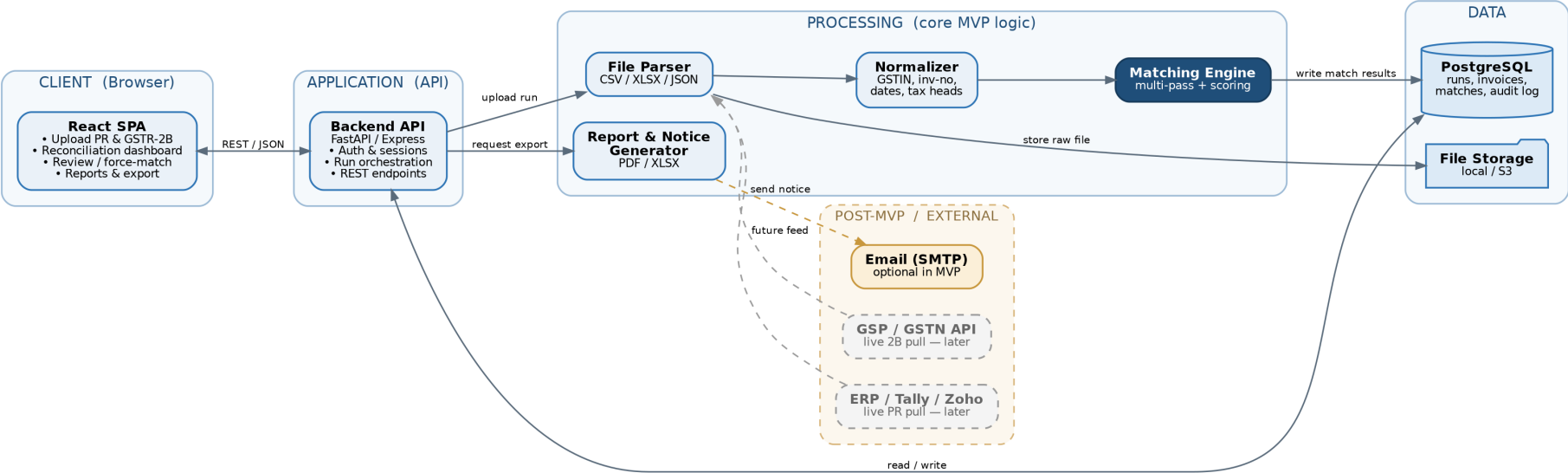


Figure 1 — MVP system architecture. Solid elements are built now; dashed amber/grey are optional or post-MVP integrations. Data flows left→right: the React SPA talks to one Backend API, which orchestrates parse → normalize → match and reads/writes PostgreSQL.

2. Reconciliation Workflow

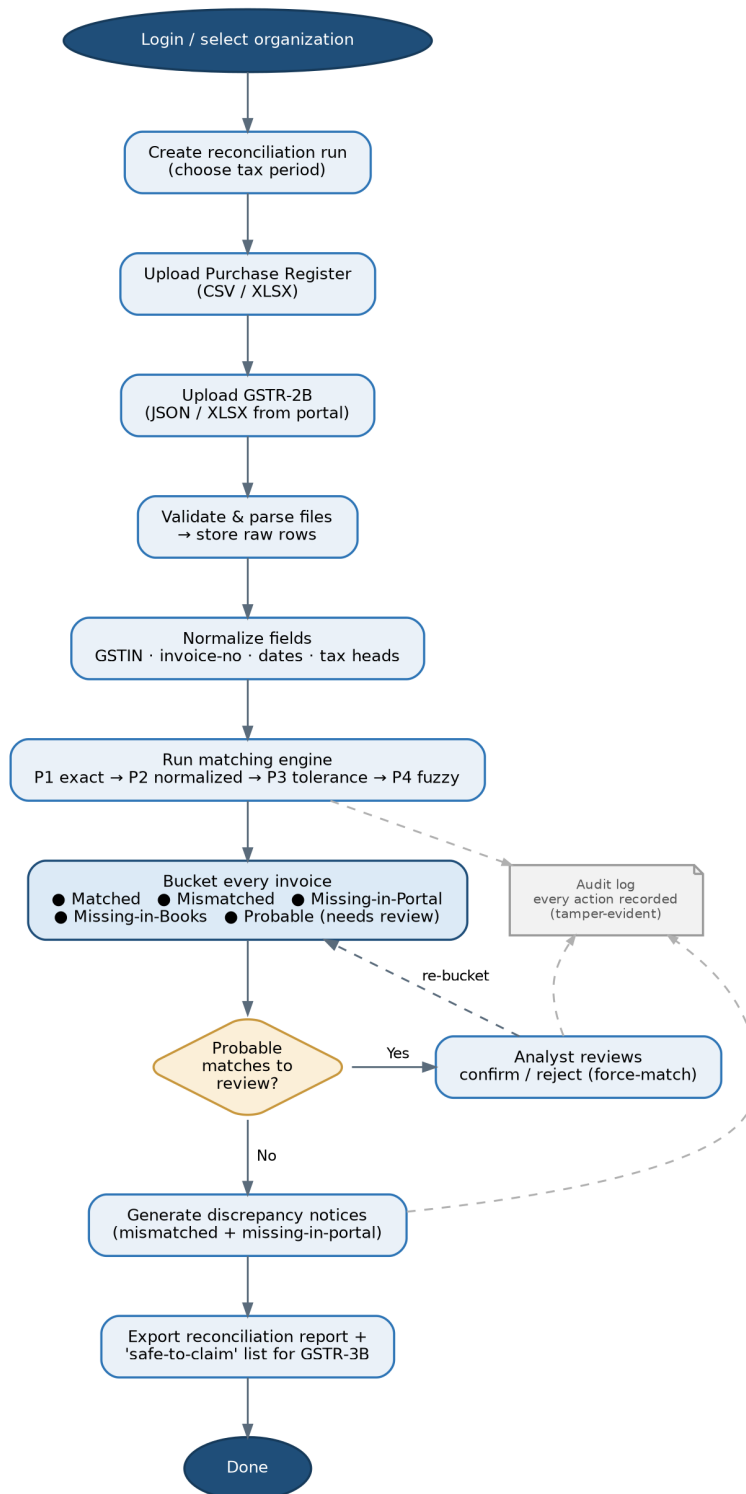


Figure 2 — End-to-end flow for one tax period, from upload to an exportable, GSTR-3B-ready result. Every state-changing step writes to the audit log.

3. ER Diagram — data model

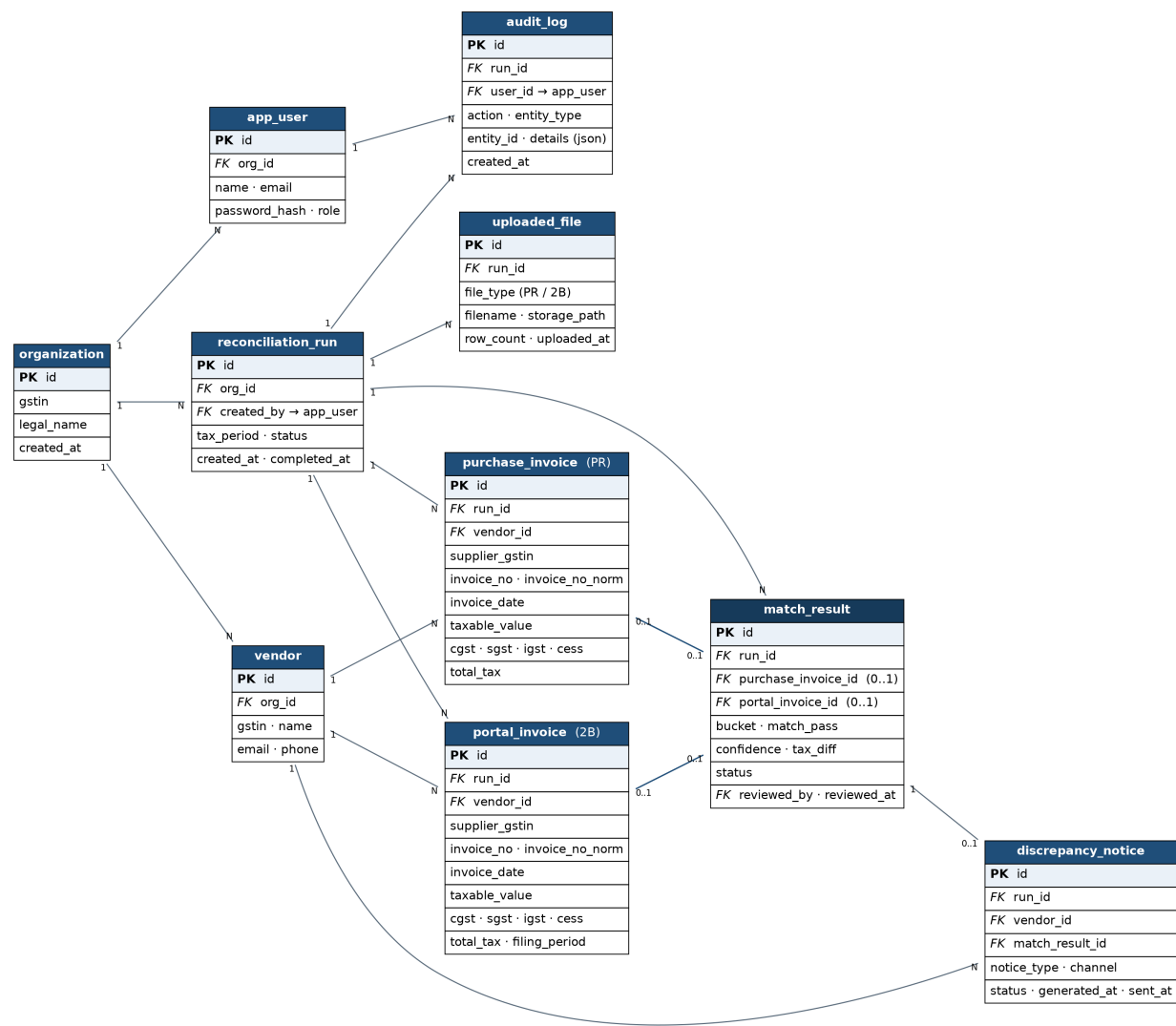


Figure 3 — Relational schema (PostgreSQL). match_result is the heart of the system: each row links at most one purchase_invoice and one portal_invoice and carries the bucket, confidence and tax difference.

4. Build Reference

4.1 Component responsibilities

Component	Responsibility in the MVP
React SPA	Upload screens, reconciliation dashboard, review / force-match UI, trigger exports.
Backend API	Auth & sessions, run lifecycle, orchestrates parse → normalize → match, serves results to the UI.
File Parser	Read CSV / XLSX / JSON, validate basic schema, persist raw rows + the original file.
Normalizer	Clean GSTIN, strip invoice-no delimiters & leading zeros, parse dates, standardize tax heads.
Matching Engine	Four-pass compare; assign each invoice a bucket, a confidence score, and a tax difference.
Report & Notice Generator	Build the reconciliation report (XLSX/PDF) and per-vendor discrepancy notices (PDF).
PostgreSQL	Persist runs, invoices, match results, notices, and the audit log.
File Storage	Retain original uploaded files for traceability (local disk in MVP, S3 later).

4.2 Workflow steps

1. Log in and select the organization.
2. Create a reconciliation run for a chosen tax period.
3. Upload the Purchase Register (CSV / XLSX).
4. Upload the GSTR-2B file (JSON / XLSX downloaded from the portal).
5. Validate & parse both files; store raw rows and the original file.
6. Normalize fields on both sides (GSTIN, invoice-no, dates, tax heads).
7. Run the matching engine: P1 exact → P2 normalized → P3 tolerance → P4 fuzzy.
8. Bucket every invoice: Matched · Mismatched · Missing-in-Portal · Missing-in-Books · Probable.
9. If there are Probable matches, an analyst confirms or rejects them (force-match), then re-bucket.
10. Generate discrepancy notices for Mismatched and Missing-in-Portal invoices.
11. Export the reconciliation report plus a 'safe-to-claim' list to feed GSTR-3B.
12. Throughout, the audit log records every state change (tamper-evident).

4.3 Core data model — key tables

Table	Purpose	Notable fields
organization	The buyer entity being reconciled.	gstn, legal_name
app_user	Login identity and audit actor.	org_id (FK), role
vendor	Supplier master, derived from invoice GSTINs.	org_id (FK), gstn, email, phone

Table	Purpose	Notable fields
reconciliation_run	One period's reconciliation job.	tax_period, status
uploaded_file	Traceability of each raw upload.	file_type, storage_path
purchase_invoice	Rows parsed from the Purchase Register.	invoice_no_norm, tax heads
portal_invoice	Rows parsed from GSTR-2B.	invoice_no_norm, filing_period
match_result	Links PR ↔ 2B; the core output.	bucket, match_pass, confidence, tax_diff
discrepancy_notice	Vendor follow-up artifact.	notice_type, channel, status
audit_log	Immutable record of every action.	action, entity_type, details

4.4 Suggested build order

1. Schema + migrations for the core tables (organization, app_user, run, both invoice tables, match_result, audit_log).
2. Auth and the create-run endpoint.
3. Upload + parser for PR and 2B — start with CSV/XLSX, then add 2B JSON.
4. Normalizer as small, pure, unit-tested functions (this is where matching quality is won or lost).
5. Matching engine Passes 1–2 (exact + normalized) producing buckets; defer tolerance & fuzzy.
6. Dashboard read views with bucket counts and drill-down.
7. Review / force-match endpoint, writing to the audit log on every change.
8. Export: reconciliation report (XLSX) first, then PDF discrepancy notices.
9. Add Pass 3 (financial tolerance) and Pass 4 (fuzzy + confidence scoring).
10. Seed demo data, then test against a real PR + 2B sample and tune thresholds.

Tip — treat **match_result** as append-only per run, and keep the normalizer deterministic and side-effect free. That keeps every claimed credit explainable and lets you re-run a period without corrupting history.